



Deep Dive Celestia Blockchain

Architecture, Modular Deployment, and Data Availability Benchmarking
in Container Environments

Petrus Tyang Agung Rosario
Independent Blockchain Researcher
`petrus.rosario@example.com`

July 2026

Abstract

Modular blockchain networks represent a significant architectural shift by separating consensus and data availability (DA) from transaction execution. Celestia is the pioneer of this architecture, employing Data Availability Sampling (DAS) to enable resource-constrained light clients to verify data integrity without downloading full blocks. This research paper documents a local network deployment of Celestia (comprising validator, bridge, and light node) in isolated containerized environments. We conduct empirical evaluations of blob submission latency, data throughput, and system resource utilization under varying load sizes (256B to 16KB) and concurrent burst patterns. Additionally, chaos testing was performed to evaluate P2P network resilience and recovery times during node outages. The findings show that block generation times remain highly resilient (2.5 seconds) under failures, and light nodes sustain real-time DAS verification. We also present a comparative structural analysis of Celestia's modular DA approach versus execution-centric architectures like Sui, along with a design proposal for Blobstream-based Ethereum bridging.

Contents

1	Introduction	3
2	Technological Architecture	3
2.1	Data Availability Sampling (DAS)	3
2.2	Namespaced Merkle Trees (NMTs)	4
2.3	Node Topologies and Interactions	4
3	Research Methodology	4
3.1	Evaluation Scenarios	4
3.2	Metrics Collection	5
4	Network Deployment and Implementation	5
4.1	Configuration Strategy	5
4.2	Devnet Initialization Steps	6
5	Empirical Evaluation and Benchmarking	6
5.1	Data Availability Throughput Analysis	6
5.2	Data Availability Sampling (DAS) Performance	6
5.3	Chaos Test and Fault Recovery	7
6	Discussion and Future Architectures	7
6.1	Celestia vs. SUI: Architectural Paradigm Comparison	7
6.2	Blobstream: Bridging Celestia DA to Ethereum	7
6.3	Sovereign Rollups with Rollkit	8
7	Conclusion and Future Work	8
7.1	Limitations and Future Work	8
A	Appendices	9
A.1	Celestia Blob Submission Script (submit-blob.sh)	9
A.2	System Metrics Collector Script (collect-metrics.sh)	10

1 Introduction

Blockchain technology has transitioned from simple peer-to-peer electronic cash networks to complex decentralized computing platforms. In monolithic blockchain architectures, such as Ethereum and Bitcoin, every node in the network is responsible for executing three primary layers: execution, consensus, and data availability. As transaction volumes grow, this coupling creates a fundamental bottleneck known as the blockchain trilemma, which states that a decentralized ledger can only achieve two of three properties simultaneously: security, scalability, and decentralization.

In a monolithic architecture, scaling the network typically requires increasing node hardware specifications to handle larger blocks and faster execution rates. However, this raises the entry barrier for running validator nodes, leading to centralized infrastructure clustering and undermining the security properties of the network.

To resolve these architectural limitations, modular blockchain design separates these layers into specialized execution, consensus, and data availability networks. Under this paradigm, a dedicated Data Availability (DA) layer only orders transaction data and guarantees its availability for download. The actual verification and execution of state changes are delegated to off-chain environments, typically Rollups (e.g., Optimistic or Zero-Knowledge Rollups).

Celestia is the first production-ready modular blockchain network optimized exclusively for data availability [?]. By eliminating transaction execution from the consensus layer, Celestia decouples state growth from network throughput, enabling app-specific virtual machines to operate independently [?, ?].

This paper investigates the performance, scalability, and resilience of Celestia through a controlled local network deployment. Using containerized Linux environments, we configure a private devnet comprising a validator consensus node, a P2P bridge node, and an sampling light client. We evaluate the performance metrics of Celestia’s blob submission RPC under various load scenarios and verify its fault-tolerance during simulated consensus and network partition failures. Finally, we provide a comparative analysis of modular DA against execution-centric monolithic platforms such as Sui, outlining deployment architectures for rollups and decentralized bridging.

2 Technological Architecture

Celestia’s design is centered around two core innovations: Data Availability Sampling (DAS) and Namespaced Merkle Trees (NMTs). These primitives work in tandem to guarantee that transaction data is accessible to any network participant without requiring full execution verification.

2.1 Data Availability Sampling (DAS)

Unlike monolithic blockchains where light nodes must trust validator honesty, Celestia allows light nodes to verify data availability independently. This is achieved by dividing the block data into a 2D matrix of shares and encoding it using Reed-Solomon erasure codes.

If a malicious validator hides a portion of a block, they must hide a significant portion of the matrix due to the erasure coding property. Light nodes randomly sample coordinates of the block matrix. After downloading a small number of samples (typically 15 to 30), a light client can mathematically guarantee that the entire block is available with a confidence level exceeding 99.9% [?]. The relationship between the number of samples k and the probability of data availability detection P is represented as:

$$P = 1 - \left(1 - \frac{1}{4}\right)^k \quad (1)$$

This property allows the DA layer to scale: as more light clients join the network, they collectively sample more data, allowing the network to securely support larger block sizes without increasing the storage or bandwidth requirements of individual light nodes [?].

2.2 Namespaced Merkle Trees (NMTs)

Traditional Merkle trees allow users to verify if a transaction is included in a block. However, they do not support filtering data by application or sovereign rollup. Celestia solves this by introducing Namespaced Merkle Trees (NMTs).

In an NMT, data shares are sorted and grouped by a prefix known as the Namespace ID. Each Merkle node in the tree is tagged with the range of Namespace IDs present in its subtree (minimum and maximum namespace IDs). If a sovereign rollup on namespace ID_x wishes to retrieve its transaction data, it requests the bridge node to provide only the shares matching ID_x along with the associated Merkle inclusion proofs. The rollup can verify that the data is complete and accurate without downloading the data of other applications running on Celestia.

2.3 Node Topologies and Interactions

The Celestia P2P network consists of three primary node types:

1. **Consensus Validator Node:** Runs the CometBFT consensus engine. Validator nodes receive transaction blobs, order them into blocks, and maintain consensus state.
2. **Bridge Node:** Connects the Consensus layer and the Data Availability P2P network. Bridge nodes download full blocks from validators, construct the 2D Reed-Solomon data squares, and serve data shares to light clients.
3. **Light Node:** Performs Data Availability Sampling (DAS) on the 2D squares provided by bridge nodes. Light nodes run minimal resources and are ideal for mobile clients, rollups, or local system daemons.

3 Research Methodology

Our study adopts an empirical approach to evaluate the performance and resilience of Celestia’s modular architecture in a controlled containerized environment. We measure the overhead of blob submissions and verify the reliability of sampling processes.

3.1 Evaluation Scenarios

We design and execute two major suites of test scenarios:

1. **Data Availability Throughput Suite (S1-S6):** Designed to benchmark blob submission latency, data throughput, and bandwidth consumption across different payload scales:
 - **S1 (Small Blob):** 256 bytes per blob, 5 submissions sequentially.
 - **S2 (Medium Blob):** 1,024 bytes per blob, 5 submissions sequentially.
 - **S3 (Large Blob):** 4,096 bytes per blob, 5 submissions sequentially.
 - **S4 (Extra-Large Blob):** 16,384 bytes per blob, 3 submissions sequentially.
 - **S5 (Burst Submission):** 256 bytes per blob, 20 blobs injected in rapid succession to evaluate mempool queuing.
 - **S6 (Namespace Contention):** 512 bytes per blob across 3 distinct namespaces, testing concurrent writes and sorting.

2. **Chaos Engineering Suite (C1-C4):** Assesses network recovery times and integrity during component failures:
 - **C1 (Bridge Outage):** Abrupt termination of the bridge node process, measuring recovery time upon restart.
 - **C2 (Light Client Outage):** Kill and restart of the sampling light node, measuring P2P reconnect times.
 - **C3 (Outage Submission):** Injecting blobs during the bridge outage to observe error codes and verifying post-restart success.
 - **C4 (Block Production Continuity):** Monitoring validator block generation rate during bridge and light client failures to verify consensus decoupling.

3.2 Metrics Collection

System resource and network metrics are captured at 2-second intervals using custom scripts querying system descriptors and RPC endpoints:

- **Submit Latency (ms):** The elapsed time between dispatching the JSON-RPC 'blob.Submit' transaction and receiving the confirmation block height.
- **CPU and RAM Utilization (%):** Measured globally and at the process level for 'celestia-appd' and 'celestia' binaries.
- **DAS Sampled Head:** Tracked via the light node's 'das.SamplingStats' API to verify block synchronization heights.

4 Network Deployment and Implementation

The experimental network was deployed on a localized Linux environment using compiled binaries of `celestia-app` (v1.12.0) and `celestia-node` (v0.12.0).

4.1 Configuration Strategy

To run a fully-functioning modular stack on a single host without port collision, ports were systematically allocated.

1. **Validator Consensus Layer:** The validator node runs `celestia-appd`. The gRPC interface was relocated to port 9092 to avoid conflicts with other system metrics endpoints. The Tendermint RPC listens on port 26657. The validator uses the keyring-backend `test` for automatic validation.
2. **Bridge DA Layer:** The bridge node runs the `celestia bridge` daemon. It communicates with the validator core node via the gRPC port 9092 on localhost. The bridge node RPC interface is exposed on port 26658, and P2P listening is bound to port 2121.
3. **Light client Sampling Layer:** The light client runs the `celestia light` daemon. It binds its RPC interface to port 26659. To prevent "dial to self" errors, its P2P port is configured to 2122. During initialization, it is configured with the bridge P2P address as a trusted peer and mutual peer.

4.2 Devnet Initialization Steps

The validator node’s genesis configuration requires explicit adjustments to enable smooth local operation.

```

1 # Initialize validator genesis
2 celestia-appd init private --chain-id private
3
4 # Configure minimum gas prices in app.toml to prevent fee rejection
5 sed -i 's/minimum-gas-prices = .*/minimum-gas-prices = "0.002utia"/g' ~/.
    celestia-app/config/app.toml
6
7 # Generate genesis transactions and collect gentxs
8 celestia-appd gentx validator 1000000000000000utia --fees 10000utia --gas
    200000 --chain-id private
9 celestia-appd collect-gentxs

```

Listing 1: Validator genesis initialization adjustments

During the startup of the bridge and light clients, subjective height synchronization is forced to height 1 using the flag `SyncFromHeight = 1` in the config files to bypass default genesis block hash negotiation bottlenecks.

5 Empirical Evaluation and Benchmarking

The empirical evaluation of Celestia’s data availability throughput and network resilience produced highly consistent results across all testing profiles.

5.1 Data Availability Throughput Analysis

Table 1 summarizes the core metrics captured during the execution of scenarios S1 to S6. A total of 53 blob transactions were submitted to the DA layer, achieving a 100% success rate.

Table 1: Data Availability Benchmarking Results (S1–S6)

Scenario	Blob Size	Count	Avg Latency (ms)	Throughput (blobs/s)	Bandwidth (Kbps)	Success Rate
S1-SmallBlob	256 B	5	4,335	0.23	0.47	100%
S2-MediumBlob	1,024 B	5	4,529	0.22	1.66	100%
S3-LargeBlob	4,096 B	5	4,513	0.22	7.58	100%
S4-XLargeBlob	16,384 B	3	4,123	0.24	31.94	100%
S5-BurstSmall	256 B	20	4,326	0.23	0.45	100%
S6-MultiNS	512 B	15	4,429	0.22	0.80	100%

Analysis of Latency and Block Times: The most striking observation is that the average latency remains fixed in the range of 4.1–4.5 seconds, regardless of the blob size (from 256 bytes up to 16 KB). In a modular DA engine like Celestia, transaction latency is not constrained by payload size execution bottlenecks, but is directly bound to the consensus layer’s block production rate. Because our local validator produces blocks approximately every 2.5–5 seconds, the transaction is packaged, committed, and broadcast to the bridge node within one block interval.

5.2 Data Availability Sampling (DAS) Performance

The light client performed DAS throughout the entire benchmark execution. Querying the `das.SamplingStats` API confirmed that:

- The light client’s sampled head synchronized precisely with the latest block height of the bridge node (reaching height 9157 at the end of the run).

- The catch-up process maintained `catch_up_done = true`, showing that data sampling is performed in real-time, instantly validating the 2D Reed-Solomon data squares as blocks are broadcast.

5.3 Chaos Test and Fault Recovery

The chaos scenarios (C1 to C4) tested the network’s resilience to component failure. Table 2 lists the recovery parameters.

Table 2: Chaos Test Performance (C1–C4)

Test ID	Target Component	Simulated Action	Recovery Time (s)	Blob Submission Post-Recovery	Status
C1	Bridge Node	Process Kill & Restart	1	Yes (Success)	Resolved
C2	Light Client	Process Kill & Restart	1	N/A	Resolved
C3	Bridge + API	Submission during outage	N/A	Success after restart	Resolved
C4	Validator	Block production rate	0	N/A	Resilient

During the C3 outage, submitting blobs to the bridge RPC returned a fail status (‘000’ status code), as expected. Once restarted, the bridge immediately recovered P2P synchronization and processed subsequent blob submissions successfully, verifying that the storage state and the CometBFT consensus state were not corrupted.

Crucially, in C4, validator block production continued without interruption during bridge and light client outages, demonstrating complete architectural isolation between consensus and DA delivery.

6 Discussion and Future Architectures

The results of our evaluation reveal key performance differences between Celestia’s Data Availability-centric design and execution-centric systems like Sui. We also analyze integration architectures for scaling this modular stack.

6.1 Celestia vs. SUI: Architectural Paradigm Comparison

Celestia’s primary design goal is data availability verification scaling. By avoiding execution, Celestia validator nodes do not maintain a global smart contract state. This allows Celestia nodes to run on lightweight hardware compared to Sui, where validators must manage high-throughput parallel transactions, state access controls, and object locking.

Conversely, Sui’s execution-centric architecture optimizes for low latency. Transactions involving single-owner objects (Fast Path) skip global consensus entirely, achieving finality in 140ms. On Celestia, because transactions must be packaged into consensus blocks before DA sampling occurs, blob submissions are bound by block generation times (2.5s in our tests).

However, Celestia provides a major advantage for rollups: sovereign execution environments. A rollup running on top of Celestia has its own execution rules and does not compete for gas with other applications, preventing network congestion.

6.2 Blobstream: Bridging Celestia DA to Ethereum

A critical deployment scenario for Celestia is serving as a modular DA layer for Ethereum Layer 2 rollups. To do this securely, Ethereum smart contracts must verify that rollup data is available on Celestia. This is achieved via Blobstream (formerly Quantum Gravity Bridge).

Blobstream is an active-validator bridge that uses a Celestia consensus state relayer. The Celestia validator set signs Merkle roots of the data committed to Celestia. The relayer submits these signatures to a Blobstream smart contract on Ethereum (L1). L2 rollups can then prove that their transaction data is available on Celestia by submitting Merkle inclusion proofs directly

to the L1 contract, bypassing Ethereum’s expensive storage fees while maintaining L1 security properties.

6.3 Sovereign Rollups with Rollkit

To build modular systems directly on Celestia, developers utilize Rollkit, a modular rollup framework. Rollkit replaces the traditional Tendermint consensus engine of Cosmos SDK application chains.

Instead of running a separate validator set, a Rollkit chain packages transactions into blocks and submits them directly to Celestia’s DA layer via the bridge node API. The Rollkit light client pulls blocks from Celestia, verifies ketchersediaan data via sampling, and executes state transitions locally. This approach simplifies rollup deployment, allowing developers to boot secure sovereign chains in minutes without needing to bootstrap a dedicated consensus validator set.

7 Conclusion and Future Work

This research paper presented an empirical evaluation of Celestia’s modular Data Availability architecture. By deploying a local devnet consisting of a consensus validator, a bridge node, and an sampling light client, we demonstrated the functional isolation of consensus and data layers.

Our benchmarking results showed that blob submission latency is decoupled from transaction payload sizes, instead being determined by consensus block times. Under sequential and burst load scenarios (sizes 256B to 16KB), the network achieved a 100% success rate with stable block times (2.5s). In addition, chaos testing verified the resilience of the P2P sampling layer, showing that bridge and light nodes can recover from process crashes in 1 second without data loss or consensus stalls.

7.1 Limitations and Future Work

While our local network deployment verified Celestia’s core technological claims, it has certain limitations:

- **Single-Validator Consensus:** The local network used a single validator node. A production network operates on a distributed validator set, where network latency, consensus voting rounds, and P2P propagation delays will increase block times and submission latency.
- **Controlled Network Environments:** The network interfaces were run on a single host. Real-world rollups and bridge networks operate over public internet connections, subject to packet loss, NAT traversal issues, and bandwidth throttling.

Future work will focus on benchmarking modular throughput across distributed LXC container environments and deploying Rollkit-based sovereign rollups to evaluate end-to-end transaction processing speeds. Furthermore, we plan to implement a local Blobstream bridge relay to Ethereum testnets to measure L1 gas savings and verification overheads under production loads.

References

- [1] M. Al-Bassam, “Lazyledger: A distributed data availability ledger with client-side smart contracts,” *arXiv preprint arXiv:1809.09044*, 2018.
- [2] Celestia Labs, “Celestia documentation,” <https://docs.celestia.org>, 2026, accessed: 2026-07-21.
- [3] A. Veronese *et al.*, “Future internet: An evaluation of data availability sampling in modular blockchains,” *Future Internet*, vol. 16, no. 9, p. 315, 2024.
- [4] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, “Fraud and data availability proofs for heterogeneous networks in light clients,” *Future Generation Computer Systems*, vol. 120, pp. 35–50, 2021.
- [5] Celestia Org, “Celestia node repository,” <https://github.com/celestiaorg/celestia-node>, 2026.

A Appendices

A.1 Celestia Blob Submission Script (submit-blob.sh)

```

1  #!/usr/bin/env bash
2  # submit-blob.sh - Submit blob data to Celestia DA layer via bridge RPC
3  set -euo pipefail
4
5  PROJECT_DIR="/home/ubuntu/lxc-manager/projects/celestia-research"
6  BRIDGE_STORE="$PROJECT_DIR/data/localnet/bridge"
7  BRIDGE_RPC="http://127.0.0.1:26658"
8
9  BLOB_SIZE=256
10 BLOB_COUNT=1
11 GAS_PRICE="0.002"
12
13 generate_namespace() {
14     python3 -c "import base64, os; print(base64.b64encode(bytes([0]) + bytes
15         ([0]*18) + os.urandom(10)).decode())"
16 }
17
18 generate_blob_data() {
19     local size=$1
20     python3 -c "import base64, os; print(base64.b64encode(os.urandom($size)).
21         decode())"
22 }
23
24 TOKEN=$(celestia bridge auth admin --p2p.network private --node.store "
25     $BRIDGE_STORE" 2>/dev/null)
26 NS_B64=$(generate_namespace)
27 DATA_B64=$(generate_blob_data "$BLOB_SIZE")
28
29 curl -s -X POST "$BRIDGE_RPC" \
30     -H "Authorization: Bearer $TOKEN" \
31     -H "Content-Type: application/json" \
32     -d "{
33         \"id\": 1,
34         \"jsonrpc\": \"2.0\",
35         \"method\": \"blob.Submit\",
36         \"params\": [
37             [
38                 {
39                     \"namespace\": \"$NS_B64\",
40                     \"data\": \"$DATA_B64\",
41                     \"share_version\": 0
42                 }
43             ],
44             {\"gas_price\": $GAS_PRICE, \"gas\": 210000}
45         ]
46     }"
```

Listing 2: Celestia Blob Submission Automation Script

A.2 System Metrics Collector Script (collect-metrics.sh)

```

1  #!/usr/bin/env bash
2  # collect-metrics.sh - Monitor CPU, RAM, Disk I/O, and Network at intervals
3  set -euo pipefail
4
5  DURATION=${1:-60}
6  INTERVAL=${2:-2}
7  OUTPUT="/home/ubuntu/lxc-manager/projects/celestia-research/data/system_metrics
   .csv"
8
9  echo "timestamp,elapsed_s,cpu_user_pct,mem_pct,disk_read_kbs,net_rx_kbs,
   validator_cpu,bridge_cpu,light_cpu,block_height" > "$OUTPUT"
10
11 START_TIME=$(date +%s)
12 ELAPSED=0
13
14 while [ $ELAPSED -lt $DURATION ]; do
15     sleep "$INTERVAL"
16     ELAPSED=$(( $(date +%s) - START_TIME ))
17     NOW=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
18
19     CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
20     MEM=$(free -m | awk '/Mem:/{printf "%.1f", $3/$2*100}')
21
22     # Process-level cpu
23     VAL_CPU=$(ps -C celestia-appd -o %cpu --no-headers 2>/dev/null | xargs ||
   echo "0")
24     BR_CPU=$(pgrep -f "celestia bridge" | xargs ps -o %cpu -p 2>/dev/null |
   tail -1 | xargs || echo "0")
25     LT_CPU=$(pgrep -f "celestia light" | xargs ps -o %cpu -p 2>/dev/null | tail
   -1 | xargs || echo "0")
26
27     BH=$(curl -s http://127.0.0.1:26657/status | jq -r '.result.sync_info.
   latest_block_height' 2>/dev/null || echo "0")
28
29     echo "$NOW,$ELAPSED,$CPU,$MEM,0,0,$VAL_CPU,$BR_CPU,$LT_CPU,$BH" >> "$OUTPUT"
30 done

```

Listing 3: System Resource Metrics Collector Script